visualML

A reverse-mode autodiff engine, its optimizers, and running it in the browser

Roberto Nsoni

October 12, 2026

Abstract

visualML is a small machine-learning engine written from scratch on top of NumPy: a tensor with reverse-mode automatic differentiation, layers, losses, first-order optimizers (SGD, momentum, RMSProp, Adam) and second-order ones (BFGS and damped Newton with a finite-difference Hessian) under a strong-Wolfe line search. The same package runs unmodified in the browser through Pyodide, where a website uses it to train and inspect networks live. This document derives every operation and update rule exactly as implemented, describes how the implementation is verified, and reports results on MNIST.

1 Introduction

Most people who train neural networks never see the machinery that makes it possible: the graph that records every operation, the chain rule applied backwards through it, the update rule that turns a gradient into a step. visualML is that machinery written out, small enough to be read in an afternoon, and complete enough to reach the accuracy one expects on MNIST (Section 8).

The project has two halves that share one codebase.

- **The engine** (`src/ml/`): a `Tensor` type with reverse-mode automatic differentiation (Section 2), a set of differentiable operations (Section 3), modules and losses (Section 4), and six optimizers with a line search (Section 5). It depends on nothing but NumPy.
- **The website** (`web/`), which loads the same Python package into the browser with Pyodide (Section 7) and uses it to train networks live, and to take a trained one apart number by number.

Two rules shape the code. Every differentiable operation is checked against a numerical gradient before it is used (Section 6). And nothing the website shows is reimplemented in JavaScript: the page asks the engine, and draws what it answers.

The mathematics in this document is what the author learnt in the first two years of the Bachelor's Degree in Data Science and Engineering (GCED) at the Universitat Politècnica de Catalunya: linear algebra [6], mathematical optimisation [9], machine learning [7] and audiovisual processing [8]. The original papers and books are cited where each method appears.

2 Tensor class

2.1 Definition

A `Tensor` holds an array `data` (always `float64`), a flag `requires_grad`, and, when the flag is set, three things about how it was made: the set of tensors it was computed from (`_prev`), a closure `_backward`, and the name of the operation. Every operation on tensors returns a

new tensor, and if any input requires a gradient it records those three fields. A forward pass therefore builds a directed acyclic graph whose leaves are the inputs and the parameters, and whose root is, in training, the scalar loss L.

2.2 The backward pass

For a tensor u in the graph, $\frac{\partial L}{\partial u}$ is the gradient of the loss with respect to it: an array with the same shape as u, stored in `u.grad`. If v was computed from u and $\frac{\partial L}{\partial v}$ is already known, the chain rule gives

$$\frac{\partial L}{\partial u} = J^\top \frac{\partial L}{\partial v}, \qquad J = \frac{\partial v}{\partial u}, \tag{1}$$

with u and v read as flat vectors and J the Jacobian of v with respect to u. This product of a vector with the Jacobian is what each operation's `_backward` computes for each of its inputs, without ever building J: for an elementwise operation it is diagonal, and the product is just another elementwise operation on arrays.

A tensor used by more than one operation receives a gradient from each, and by the chain rule they add up. That is why every `_backward` does `u.grad += ...` and never `u.grad = ...`.

`Tensor.backward()` first orders the graph topologically with an iterative depth-first search, then sets $\partial L/\partial L = 1$ and calls `_backward` on every node in reverse topological order, so that each node's gradient is complete before it is passed on to its inputs.

Algorithm 1 `Tensor.backward` (reverse mode)

1: *order* ← topological order of the graph reachable from L
2: `L.grad` ← 1
3: **for** v in reverse(*order*) **do**
4: $\quad v$.`_backward`() ▷ adds its part of the gradient to the `.grad` of each input
5: **end for**

Gradients are allocated lazily (as zeros, the first time they are read) and must be reset between steps with `zero_grad()`, which every optimizer does before evaluating the loss.

2.3 Broadcasting

NumPy broadcasting lets $a \in \mathbb{R}^{n\times m}$ be added to $b \in \mathbb{R}^m$. The forward is free; the backward is not, because `b.grad` must have the shape of b. Since b was copied along the broadcast axes, its gradient is the sum of the incoming gradient over them. `reduce_grad` implements exactly this: it sums over the leading axes that broadcasting added, and over every axis where b had size 1 but the result did not.

2.4 Freezing a model

A trained model used only for predictions does not need a graph: building one keeps every intermediate array alive for a backward pass that never comes. `Module.freeze()` sets `requires_grad = False` on every parameter, after which a forward pass on a plain input records nothing. A gradient with respect to an input that does require one is still computed exactly, which is all that a saliency map needs.

3 Operations

Each operation is listed with its forward rule and the gradient it sends to each of its inputs, following (1), written on whole arrays. $a*b$ is the *elementwise* product, entry by entry (NumPy's

`a * b`); AB, with nothing between the factors, is the matrix product (`A @ B`); functions and powers of an array, such as a^p, β^a or $\frac{1}{a}$, act on each entry; and $[\,\text{condition}\,]$ is the array that is 1 where the condition holds and 0 elsewhere (a boolean array used as numbers). When an input was broadcast, its gradient is then summed over the broadcast axes by `reduce_grad` (Section 2).

3.1 Elementwise and linear algebra

operation	forward	gradient
negation	$v = -a$	$\frac{\partial L}{\partial a} = -\frac{\partial L}{\partial v}$
addition	$v = a + b$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial v}, \;\; \frac{\partial L}{\partial b} = \frac{\partial L}{\partial v}$
product	$v = a * b$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial v} * b, \;\; \frac{\partial L}{\partial b} = \frac{\partial L}{\partial v} * a$
power ($p \in \mathbb{Z}$)	$v = a^p$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial v} * p\, a^{p-1}$
exponential	$v = \beta^a$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial v} * \beta^a \ln \beta$ \quad ($\beta = e$ by default)
logarithm	$v = \log_\beta a$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial v} * \frac{1}{a \ln \beta}$
sum	$v = \sum_{\text{axes}} a$	$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial v}$ broadcast back to the shape of a
mean	$v = \frac{1}{N} \sum_{\text{axes}} a$	$\frac{\partial L}{\partial a} = \frac{1}{N} \frac{\partial L}{\partial v}$ broadcast back
transpose	$V = A^\top$	$\frac{\partial L}{\partial A} = (\frac{\partial L}{\partial V})^\top$
reshape	$V = \text{reshape}(A)$	$\frac{\partial L}{\partial A} = \text{reshape}(\frac{\partial L}{\partial V}, \text{shape}(A))$
matrix product	$V = AB$	$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial V} B^\top, \;\; \frac{\partial L}{\partial B} = A^\top \frac{\partial L}{\partial V}$

Subtraction and division are composed from these ($a - b = a + (-b)$, $a/b = a * b^{-1}$). `sum` and `mean` keep the reduced axes (`keepdims`), which makes the broadcast back in the backward pass exact; for `mean`, N is the number of elements averaged.

3.2 Activations

activation	forward	gradient
ReLU	$v = \max(0, a)$	$\dfrac{\partial L}{\partial a} = \dfrac{\partial L}{\partial v} * [\, a > 0 \,]$
sigmoid	$v = \sigma(a) = 1/(1 + e^{-a})$	$\dfrac{\partial L}{\partial a} = \dfrac{\partial L}{\partial v} * v * (1 - v)$
tanh	$v = \tanh a$	$\dfrac{\partial L}{\partial a} = \dfrac{\partial L}{\partial v} * (1 - v^2)$

Sigmoid and tanh reuse their own output v in the backward pass instead of recomputing it.

3.3 Linear layers

`Linear(n_in, n_out)` computes $Y = XW + b$ for a batch $X \in \mathbb{R}^{N \times n_{\text{in}}}$, with $W \in \mathbb{R}^{n_{\text{in}} \times n_{\text{out}}}$ and $b \in \mathbb{R}^{n_{\text{out}}}$ broadcast over the batch. It is composed of a matrix product and an addition, so its gradients come from the table above; in particular $\frac{\partial L}{\partial b_j} = \sum_n \frac{\partial L}{\partial Y_{nj}}$, the sum over the batch that `reduce_grad` does. Weights are initialised as $W_{ij} \sim \mathcal{N}(0, 2/n_{\text{in}})$ (He initialisation [1]) and biases as zero. `MLP` [7] chains `Linear` layers with an activation between consecutive ones and none after the last, which therefore outputs logits.

3.4 Convolution

`conv2d` takes $x \in \mathbb{R}^{N \times C \times H \times W}$, a filter bank $w \in \mathbb{R}^{O \times C \times k_h \times k_w}$ and a bias $b \in \mathbb{R}^{O}$, stride 1 and zero padding p. With $\tilde{x}$ the padded input, it computes the cross-correlation (as every framework does, under the name of convolution [8])

$$y_{n,o,i,j} = b_o + \sum_{c} \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} w_{o,c,u,v}\, \tilde{x}_{n,c,i+u,j+v}. \tag{2}$$

Rather than visiting every output position, the implementation loops over the $k_h k_w$ kernel offsets (u, v): for each, the shifted slice $\tilde{x}_{:,:,u:u+H_o,\, v:v+W_o}$ is contracted with the $O \times C$ matrix $w_{:,:,u,v}$ over the channel axis (one `einsum`). The gradients are

$$\frac{\partial L}{\partial w_{o,c,u,v}} = \sum_{n,i,j} \frac{\partial L}{\partial y_{n,o,i,j}}\, \tilde{x}_{n,c,i+u,j+v}, \tag{3}$$

$$\frac{\partial L}{\partial \tilde{x}_{n,c,r,s}} = \sum_{o} \sum_{u,v} \frac{\partial L}{\partial y_{n,o,r-u,s-v}}\, w_{o,c,u,v}, \tag{4}$$

$$\frac{\partial L}{\partial b_o} = \sum_{n,i,j} \frac{\partial L}{\partial y_{n,o,i,j}}, \tag{5}$$

where the second sum only has the terms for which $(r - u, s - v)$ is a position of the output: an input pixel receives a gradient from every output that used it. The code computes them with the same loop over (u, v) as the forward pass, adding each offset's share into `x.grad`, and finally cuts the padding off.

3.5 Max pooling

`maxpool2d(k)` reshapes each $k \times k$ block of the input into its own axes and takes the maximum over them, $y_{n,c,i,j} = \max_{0 \le u,v < k} x_{n,c,ki+u,\,kj+v}$. Writing $\mathrm{up}(\cdot)$ for repeating every value k times along both spatial axes (so that it covers its block again), the gradient is

$$\frac{\partial L}{\partial x} = \mathrm{up}\Big(\frac{\partial L}{\partial y}\Big) * \big[\, x = \mathrm{up}(y) \,\big], \tag{6}$$

which is exactly the code: `mask = x == y.repeat(k, 2).repeat(k, 3)`, times the gradient repeated the same way. It flows only to the position that won each block. If two values of a block tie for the maximum, both receive it; with real-valued inputs this has probability zero. A max-pool has no parameters.

4 Losses

4.1 Mean squared error

$L = \frac{1}{N}\sum_i (\hat{y}_i - y_i)^2$, composed from subtraction, power and mean, so its gradient follows from Section 3.

4.2 Cross-entropy from logits

For logits $z \in \mathbb{R}^{N \times K}$ and integer labels $y \in \{0, \dots, K-1\}^N$, the softmax $p_{ik} = e^{z_{ik}} / \sum_j e^{z_{ij}}$ and the cross-entropy [7] are fused into one operation:

$$L = \frac{1}{N}\sum_{i=1}^{N} \Big(- z_{i,y_i} + \log \sum_j e^{z_{ij}} \Big). \tag{7}$$

The log-sum-exp is evaluated as $m_i + \log \sum_j e^{z_{ij} - m_i}$ with $m_i = \max_j z_{ij}$, which leaves the value unchanged and keeps the exponentials from overflowing. Differentiating,

$$\frac{\partial L}{\partial z_{ik}} = \frac{1}{N}(p_{ik} - Y_{ik}), \tag{8}$$

where Y_{ik} is 1 if k is the label of example i and 0 otherwise (the one-hot labels). So the backward pass is the softmax minus the one-hot labels, divided by the batch size, without ever building the softmax into the graph. This is both cheaper and numerically safer than composing a softmax with a logarithm.

4.3 Binary cross-entropy from logits

For a single logit z and a label $y \in \{0, 1\}$, in the stable form

$$L = \frac{1}{N}\sum_i \Big(\max(z_i, 0) - z_i y_i + \log\big(1 + e^{-|z_i|}\big) \Big), \qquad \frac{\partial L}{\partial z_i} = \frac{\sigma(z_i) - y_i}{N}. \tag{9}$$

5 Optimizers

All optimizers act on the list of parameter tensors of a model. Write x_k for all parameters flattened into one vector and $g_k = \nabla f(x_k)$ for the gradient of the loss there, computed by a *closure* that zeroes the gradients, runs the forward pass and calls `backward()`. The methods and their notation follow [9, 4].

5.1 First order

With learning rate α:

$$\text{SGD:}\quad x_{k+1} = x_k - \alpha g_k, \tag{10}$$

$$\text{Momentum:}\quad v_{k+1} = \beta v_k + g_k, \qquad x_{k+1} = x_k - \alpha v_{k+1} \quad (\beta = 0.9), \tag{11}$$

$$\text{RMSProp:}\quad r_{k+1} = \rho r_k + (1-\rho) g_k^2, \qquad x_{k+1} = x_k - \alpha \frac{g_k}{\sqrt{r_{k+1}} + \varepsilon} \quad (\rho = 0.9,\ \varepsilon = 10^{-8}), \tag{12}$$

and Adam [2], momentum and RMSProp together with bias correction:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1-\beta_1) g_k, \qquad v_k = \beta_2 v_{k-1} + (1-\beta_2) g_k^2, \\ \hat{m}_k &= \frac{m_k}{1-\beta_1^k}, \qquad \hat{v}_k = \frac{v_k}{1-\beta_2^k}, \qquad x_{k+1} = x_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}, \end{aligned} \tag{13}$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. Squares and square roots are elementwise.

Learning-rate schedule. `LinearDecay` decays the rate linearly from α_0 to $0.01\,\alpha_0$ over the first k_{ld} steps and keeps it there: $\alpha_k = (1 - k/k_{ld})\,\alpha_0 + (k/k_{ld})\,0.01\,\alpha_0$ for $k \le k_{ld}$.

5.2 Line search

The second-order methods choose their step length α along a direction d with a line search satisfying the strong Wolfe conditions [4]:

$$f(x + \alpha d) \le f(x) + c_1 \alpha\, g(x)^\top d, \tag{W1}$$

$$|g(x + \alpha d)^\top d| \le c_2\, |g(x)^\top d|. \tag{W2}$$

The implementation (`blsnw32`) follows Nocedal and Wright's Algorithms 3.5 and 3.6: a bracketing phase that doubles α while f keeps decreasing, then a zoom that shrinks the bracket by bisection until both conditions hold. $c_1 = 10^{-4}$ throughout.

5.3 BFGS

BFGS keeps an approximation H_k of the *inverse* Hessian, starting from $H_0 = I$, and moves along $d_k = -H_k g_k$. After a step $s_k = \alpha_k d_k$ with change in gradient $y_k = g_{k+1} - g_k$, it updates

$$H_{k+1} = (I - \rho_k s_k y_k^\top)\, H_k\, (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top, \qquad \rho_k = \frac{1}{y_k^\top s_k}. \tag{14}$$

The Wolfe conditions guarantee $y_k^\top s_k > 0$; near the minimum both s_k and y_k vanish and the update is skipped when $y_k^\top s_k \le 10^{-12}$. The first trial step of the line search is 1, and afterwards $\alpha_{\max} = 2(f_k - f_{k-1})/(g_k^\top d_k)$ (falling back to 1 if not positive). Here $c_2 = 0.1$.

5.4 Damped Newton

Newton's method fits the local quadratic $m(p) = f + g^\top p + \tfrac{1}{2} p^\top (H + \lambda I) p$ and moves to its minimiser, $d = -(H + \lambda I)^{-1} g$, the solution of a linear system [6]. The Hessian is approximated by forward finite differences of the analytical gradient,

$$H_{:,i} \approx \frac{g(x + h e_i) - g(x)}{h}, \qquad h = 10^{-5}, \tag{15}$$

and symmetrised as $(H + H^\top)/2$, at a cost of $n + 1$ gradient evaluations. The damping starts at $\lambda = 10^{-3}$ and is multiplied by 10 until d is a descent direction ($d^\top g < 0$), which handles non-convex regions. The step length comes from the same line search with $c_2 = 0.9$. Newton and BFGS are only practical for small models, and all second-order methods require full-batch gradients: minibatch noise destroys the curvature estimate.

6 Verification

6.1 Gradient checking

Every differentiable operation is tested against a numerical gradient before it is used. For a tensor t and a function F whose output is summed to a scalar, each entry of the analytical gradient is compared with the central difference

$$\frac{\partial}{\partial t_i} \sum F(t) \approx \frac{\sum F(t + \epsilon e_i) - \sum F(t - \epsilon e_i)}{2\epsilon}, \qquad \epsilon = 10^{-6}, \tag{16}$$

with an absolute tolerance of 10^{-6} (10^{-5} for the full convolutional network). Inputs are weighted by fixed random tensors before summing, so that every output position contributes differently and a gradient routed to the wrong place cannot pass by symmetry. Besides each operation on its own, the checks cover composite graphs (a shared sub-expression, a diamond), an MLP with cross-entropy, and a complete convolutional network (convolution, ReLU, max-pool, reshape, linear, cross-entropy) with respect to every parameter. The convolution is additionally compared with a direct, loop-by-loop evaluation of its definition.

6.2 Test inventory

file	tests
`test_autograd.py` (gradient checks)	24
`test_tensor.py`	20
`test_surfaces.py`	14
`test_optim.py`	13
`test_viz.py`	11
`test_training.py`	10
`test_module.py`	8
`test_recorder.py`	8
`test_loss.py`	7
`test_linreg.py`	6
`test_linesearch.py`	5
`test_serialize.py`	3
`test_data.py`	1
total	130

Beyond gradients, linear regression trained with the engine converges to the closed-form least-squares solution (tested, and measured for every optimizer in Section 8.1), and the line search is tested against the Wolfe conditions it claims.

7 Running in the browser

The website is static: there is no server to train on. Instead, the engine itself runs in the visitor's browser. Pyodide [5] is CPython and NumPy compiled to WebAssembly; the page

downloads it once (about 15 MB), installs the visualml wheel with `micropip`, and loads a bridge module (`web/src/engine.py`) that exposes a few functions to JavaScript. Nothing about the mathematics is reimplemented in JavaScript: the page only sends inputs across and draws the arrays that come back.

Identical numbers. Running the same optimizer races and MLP training in native Python and in Pyodide produces bit-identical results (measured in `web/spike/`), which is expected: both execute the same NumPy code on IEEE 754 doubles.

Speed. Pyodide runs about 2.5–2.9× slower than native for this workload; a step of a small MLP takes about 2.4 ms, fast enough to train live inside animation frames. The page advances training a few milliseconds per frame, so the tab never blocks.

What runs where. The playground trains small MLPs live in the tab. The MNIST networks of Section 8 are trained offline with the same engine (`scripts/train_mnist.py`); the page loads their weights and computes every prediction, saliency map and intermediate activation it shows with the engine, in the tab.

8 Experiments

Two experiments: a convex problem whose answer is known in closed form, to check that the optimizers find it, and MNIST, to check that the engine trains real networks. The tables in this section are written by `make_tables.py` and by the notebook `paper/experiments.ipynb`, from files the code produced; none is typed by hand.

8.1 Linear regression against the closed form

The data is $y = Xw + \varepsilon$, with $X \in \mathbb{R}^{200\times 3}$ drawn from a standard normal, w drawn the same way and $\varepsilon \sim \mathcal{N}(0, 0.1^2)$. The model is a `Linear(3, 1)` without bias trained on the MSE, and its answer is compared with the normal equations, $w^* = (X^\top X)^{-1} X^\top y$. The Hessian of the loss is $\frac{2}{n} X^\top X$, constant, so the first-order methods use $\alpha = 1/L$ with L its largest eigenvalue, the largest step that is safe for a quadratic; Adam uses $\alpha = 0.05$. The second dataset is the same with the three features multiplied by 1, 10 and 100, which makes the problem badly conditioned.

	well conditioned		scales 1, 10, 100	
optimizer	steps to 10^{-6}	final error	steps to 10^{-6}	final error
SGD	9	9.2e-16	–	2.5e+00
Momentum	270	7.8e-12	–	1.5e+00
Adam	286	2.0e-11	379	4.5e-09
BFGS	7	9.2e-10	8	2.4e-10
Newton	2	2.9e-13	2	1.3e-13

Table 1: Linear regression with the engine against the normal equations: the number of steps until $\|w - w^*\| < 10^{-6}$ (– if never, in 500 steps for the first-order methods and 30 for BFGS and Newton) and the distance at the end. Condition numbers 1.2 and 9310.

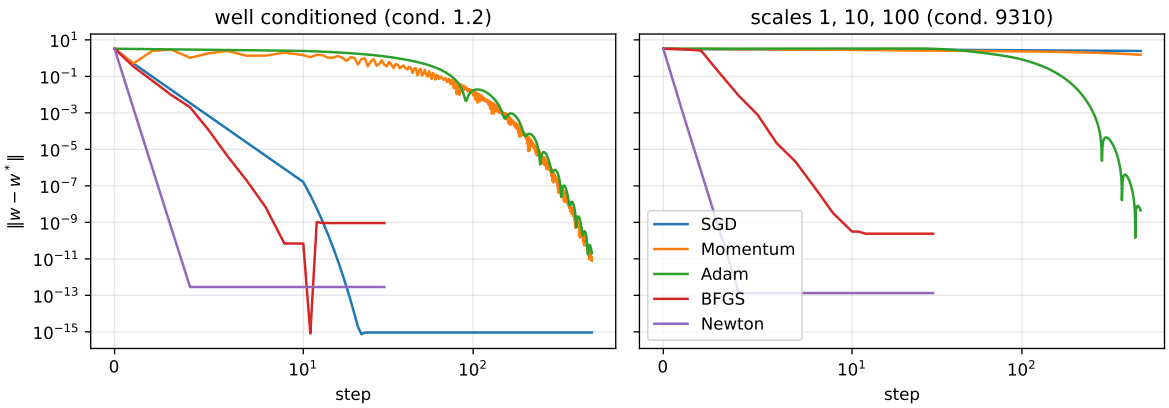


Figure 1: Distance to the closed-form solution at every step.

With a condition number near 1 every method gets there, and plain gradient descent with $\alpha = 1/L$ is among the fastest. With the features on different scales, the step that keeps the steepest direction stable is far too small for the flattest one, and SGD and momentum are still far from w^* after 500 steps. Adam rescales every coordinate by its own gradient history, which here largely undoes the scaling of the features. BFGS and Newton barely notice the conditioning: damped Newton with a finite-difference Hessian is within 10^{-6} in a couple of steps on both datasets, and BFGS needs a handful to learn the curvature.

8.2 MNIST

MNIST [3] has 60 000 training and 10 000 test images of 28×28 handwritten digits. Pixels are scaled to $[0, 1]$. Two models, both trained with the engine:

- **MLP**: $784 \to 256 \to 128 \to 10$, ReLU between layers.
- **CNN**: conv($1 \to 8$, 5×5, padding 2), ReLU, max-pool 2×2, conv($8 \to 16$, 5×5, padding 2), ReLU, max-pool 2×2, flatten, linear($784 \to 10$).

Both minimise the cross-entropy of Section 4 with Adam ($\alpha_0 = 10^{-3}$) on minibatches of 64, with the rate decaying linearly to $\alpha_0/100$ over the whole run (`LinearDecay`). Every training minibatch is augmented: each digit is scaled by $s \sim U(0.7, 1.3)$ about the centre and shifted by up to $13 - 10s$ pixels in each direction, so that it stays inside the frame, by inverse mapping and bilinear interpolation [8]; the test set is not. The MLP is trained for 15 epochs and the CNN for 10. Each model is trained with three seeds; test accuracy is measured once, at the end, on the float32 weights that are published (the ones the website loads). The *moved test* column is the same 10 000 test digits moved and resized once in the same way, with a fixed seed.

model	parameters	epochs	test accuracy (%)	per seed	moved test (%)	min / run
MLP	235,146	15	98.48 ± 0.09	98.49, 98.39, 98.56	97.38 ± 0.02	3.9
CNN	11,274	10	98.65 ± 0.17	98.85, 98.58, 98.53	97.49 ± 0.20	17.7

Table 2: MNIST test accuracy, mean $\pm$ sample standard deviation over three seeds, on the test set as published and on the same digits moved and resized once with a fixed seed (moved test). Time is CPU training time in native Python.

The convolutional network is slightly more accurate with about $21\times$ fewer parameters: weight sharing across positions builds in the translation structure that the MLP has to learn separately for every pixel.

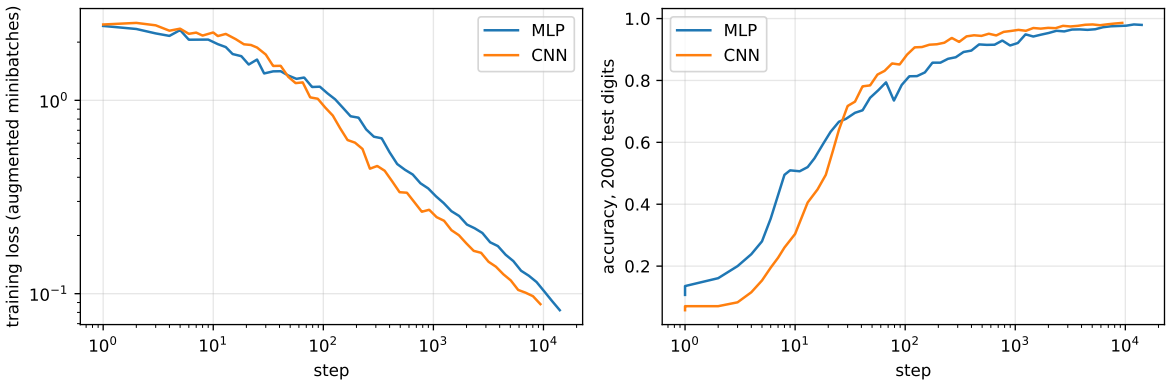


Figure 2: Training loss (on the augmented minibatches) and accuracy on a fixed slice of 2 000 test digits, at logarithmically spaced steps, for the published seed of each model.

Table 3 and Figure 3 give the test accuracy with every digit shifted sideways, for the published models and for the same CNN trained without augmentation.

	horizontal shift (pixels)							
model	−6	−4	−2	0	+2	+4	+6	moved test
CNN, no augmentation	47.9	75.5	96.8	99.0	97.4	81.7	35.7	81.98
CNN (published)	93.2	97.5	98.4	98.9	98.5	97.2	88.9	97.72
MLP (published)	84.1	96.7	98.3	98.5	98.3	95.9	82.4	97.38

Table 3: MNIST test accuracy (%) with every digit moved sideways, and on the moved-and-resized test set of Table 2. The CNN without augmentation is the same network, seed and schedule as the published one, trained on the digits as they are; one seed.

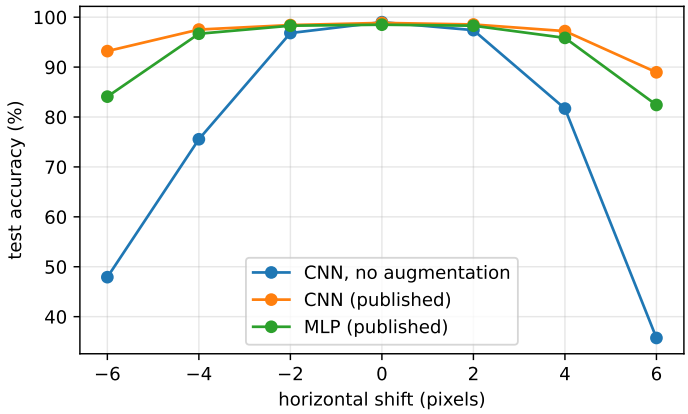


Figure 3: Test accuracy against a horizontal shift of every digit.

9 Reproducibility

Everything in this document can be regenerated from the repository.

```
pytest -q                                  # 130 tests, including every gradient check
python scripts/get_mnist.py                # MNIST into data/raw/mnist, md5-checked
python scripts/train_mnist.py --report     # both models x 3 seeds -> web/public/recognize/
```

```
cd paper && python make_tables.py          # the MNIST table of section 8 from those files
jupyter nbconvert --to notebook --execute --inplace experiments.ipynb
                                           # linear regression, curves, shifts (~20 min)
latexmk -pdf main.tex
```

Training is deterministic given the seed. The published weights, the training logs and the per-seed results are in `web/public/recognize/`.

References

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015.

[2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[3] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[4] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2 edition, 2006.

[5] The Pyodide development team. Pyodide: Python in the browser with WebAssembly. `https://pyodide.org`, 2021.

[6] Universitat Politècnica de Catalunya. àlgebra (ALG). Lecture notes, Bachelor's Degree in Data Science and Engineering (GCED).

[7] Universitat Politècnica de Catalunya. Aprenentatge automàtic 1 (AA1). Lecture notes, Bachelor's Degree in Data Science and Engineering (GCED).

[8] Universitat Politècnica de Catalunya. Introducció al processament audiovisual (IPA). Lecture notes, Bachelor's Degree in Data Science and Engineering (GCED).

[9] Universitat Politècnica de Catalunya. Optimització matemàtica (OM). Lecture notes, Bachelor's Degree in Data Science and Engineering (GCED).